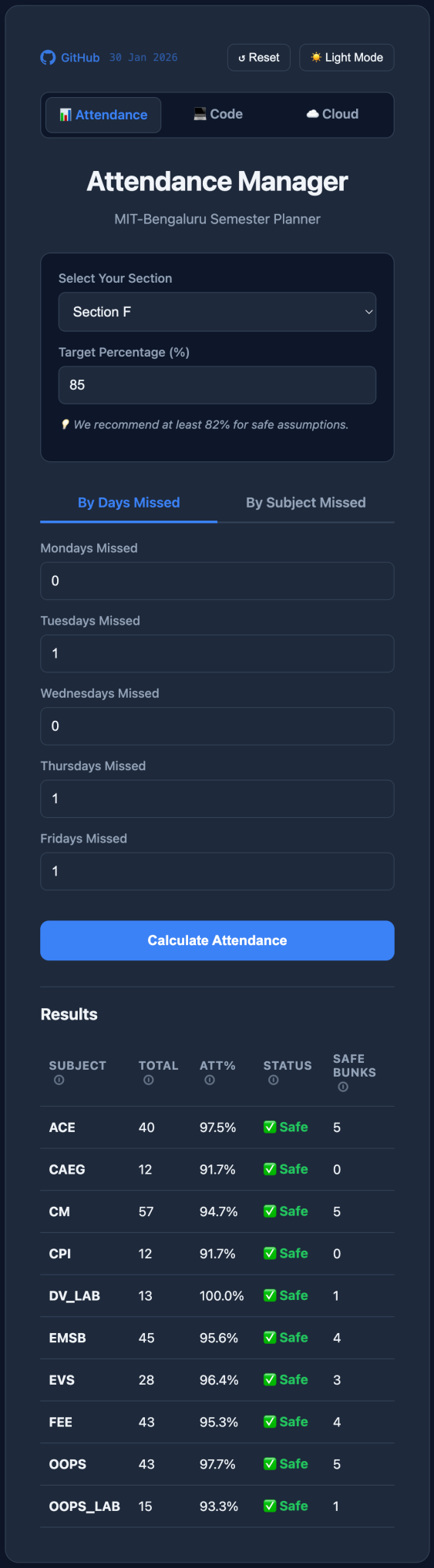


WEBSITE MANUAL & INFO

1. Dynamic Section-Based Calculation

Unlike generic calculators, this system utilizes a specific database for every section (A–X). It recognizes that schedule loads vary; for instance, Section A may have 45 hours of a subject while Section B has 40. The algorithm dynamically loads the correct class limits based on the user's selection to ensure the baseline data is accurate.

2. Calendar-Aware Day Aggregation

To prevent estimation errors, the system calculates the exact number of specific working days (e.g., total Mondays vs. total Fridays) remaining in the semester. This "day-specific" counting is crucial for subjects that only occur once or twice a week, ensuring the projected attendance percentage is mathematically precise.

3. The "Safe Bunk" Algorithm

Using the total class count and the user’s current attendance, the system performs a predictive calculation. It outputs the exact number of classes a student can afford to miss ("Safe Bunks") while strictly maintaining their desired target percentage.

4. The 82% Safety Buffer

The system defaults to a target of 82% rather than the standard 75%. This recommended buffer accounts for potential real-world discrepancies, such as:

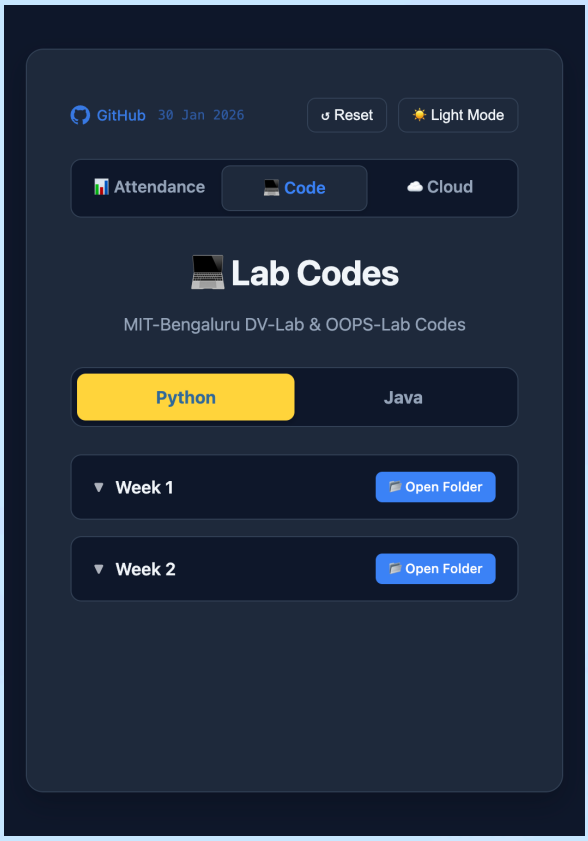
- Teacher errors in manual roll calls.
- Attendance being taken during a specific hour where the student might have been briefly absent.
- Last-minute schedule swaps between faculty.

5. State Management & Customization

- Theme Preferences: Includes a toggle for Light/Dark mode to suit user preference, which persists across sessions.
- Hard Reset: A dedicated reset function clears all Local Storage data, reverting custom targets and attendance logs to factory defaults for a fresh start.

6. Open Source Transparency

The project is linked directly to a GitHub Repository, allowing for full transparency. This enables other developers and students to audit the calculation logic, report bugs, suggest features, or contribute to the source code.



1. Centralized Lab Resources

This page serves as a dedicated archive for Data Visualization (DV) and Object-Oriented Programming (OOPS) lab codes. It eliminates the need for students to constantly ask peers for file copies by providing a single, reliable source of truth.

2. Direct View & Download

The interface features a dual-action system for every file:

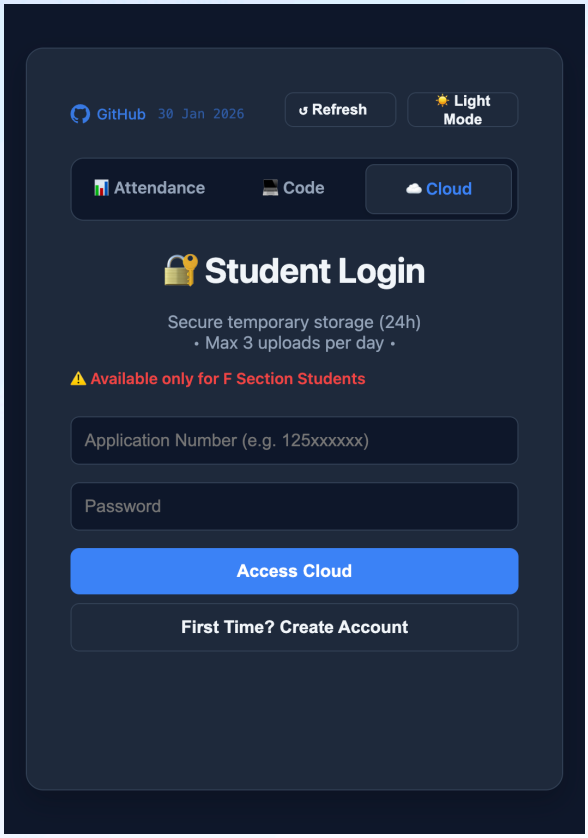
- "View": Opens the code directly in GitHub for quick reading and copying.
- "DL" (Download): Instantly downloads the raw file to the user's local machine, ready for execution.

3. Organized Folder Structure

To prevent clutter, the repository uses an accordion-style UI. Files are grouped by "Week" (e.g., Week 1, Week 2). Users can expand a specific week to find the exact file they need, keeping the interface clean and navigable.

4. Multi-Language Support

A tabbed interface allows seamless switching between Python and Java repositories. This ensures students can quickly locate language-specific assignments without scrolling through irrelevant data.



1. Secure Student Login

Access is restricted to authorized students via a whitelist-based authentication system. Students log in using their Application Number, ensuring only verified users (all CSE 2025 Student Batch) can access the storage facility.

2. Daily Upload Limits

To prevent spam and abuse, the system enforces a strict 3-upload daily limit per user. This is enforced by a server-side Cloud Function that automatically rejects any attempt to upload a 4th file within a 24-hour window.

3. Storage Quota Management

Each student is allocated a personal 3MB storage cap. A visual progress bar tracks real-time usage (Green ➡️ Yellow ➡️ Red), giving users immediate feedback on how much space they have left before they hit the hard limit.

4. Automatic Data Cleanup

Designed for temporary storage, the system includes a 24-hour Lifecycle Policy. Files are automatically deleted from the server one day after upload, ensuring the database remains lightweight and maintenance-free.